

Boolean Network Analyzer

Parsing, simulating, and analyzing Boolean networks

Tran Phan Dang Khoi • Student ID 2352626

Faculty of Computer Science & Engineering — HCMUT

Capstone Project — Artificial Intelligence Track • 2025–2026

Agenda

What we cover today

01

Motivation

Why model with Boolean networks?

02

Theory

Influence graph, update schemes, STG, attractors

03

System design

Architecture and module overview

04

Implementation

Parser, engine, UI

05

Demo

Live walkthrough of the Streamlit app

06

Validation

Reference networks and test results

07

Conclusion

Limitations and future work

Motivation

Why Boolean networks?

ON/OFF abstraction of complex regulatory circuits

- Reproduce qualitative dynamics when quantitative parameters are unknown
- Cell-cycle, signal-transduction, gene regulatory networks
- Synthetic biology design and analysis
- Discrete state space — exhaustively analysable for small n

But: 2^n curse of dimensionality

A 20-node network has $\sim 10^6$ states; 30-node has $\sim 10^9$. Manual inspection is impossible — tooling is essential.

2^n

states for n nodes

$n = 10 \rightarrow 1,024$

$n = 20 \rightarrow \sim 1 \text{ million}$

$n = 30 \rightarrow \sim 1 \text{ billion}$

Boolean networks — formal definition

A Boolean network is a tuple (V, F)

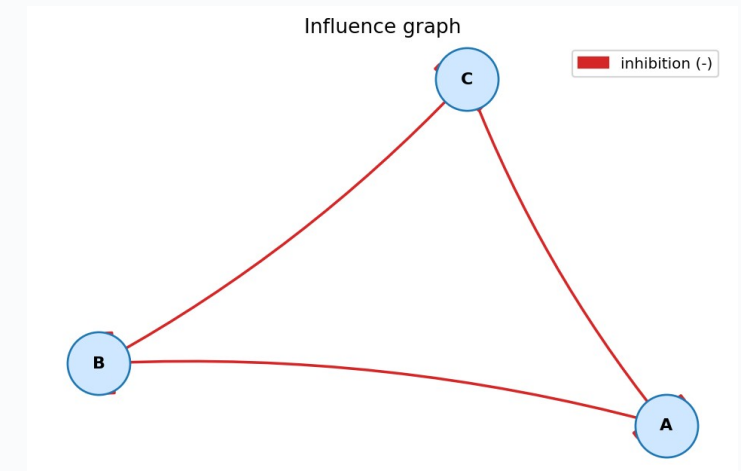
- $V = \{x_1, \dots, x_n\}$ — finite set of Boolean variables (genes, proteins, ...)
- $F = \{f_1, \dots, f_n\}$ — Boolean update rules: $f_i : \{0,1\}^n \rightarrow \{0,1\}$
- State at time t : $s(t) \in \{0,1\}^n$
- Dynamics: $s(t+1)$ determined by F and the chosen update scheme

Example — bistable toggle switch (.bnet)

targets, factors

A, !B

B, !A



Influence graph of the
3-gene repressilator

Update schemes

Which nodes update at each time step?

Synchronous

- Every node updates simultaneously
- $s_i(t+1) = f_i(s(t))$ for all i
- Fully deterministic — out-degree 1 in the STG
- Original Kauffman (1969) formulation
- Simple to analyse, but biologically idealised

Asynchronous

- Exactly one node updates per step
- Choice is non-deterministic among nodes whose rule disagrees
- STG branches — out-degree ≥ 1
- Self-loop on every fixed point
- Biologically more realistic

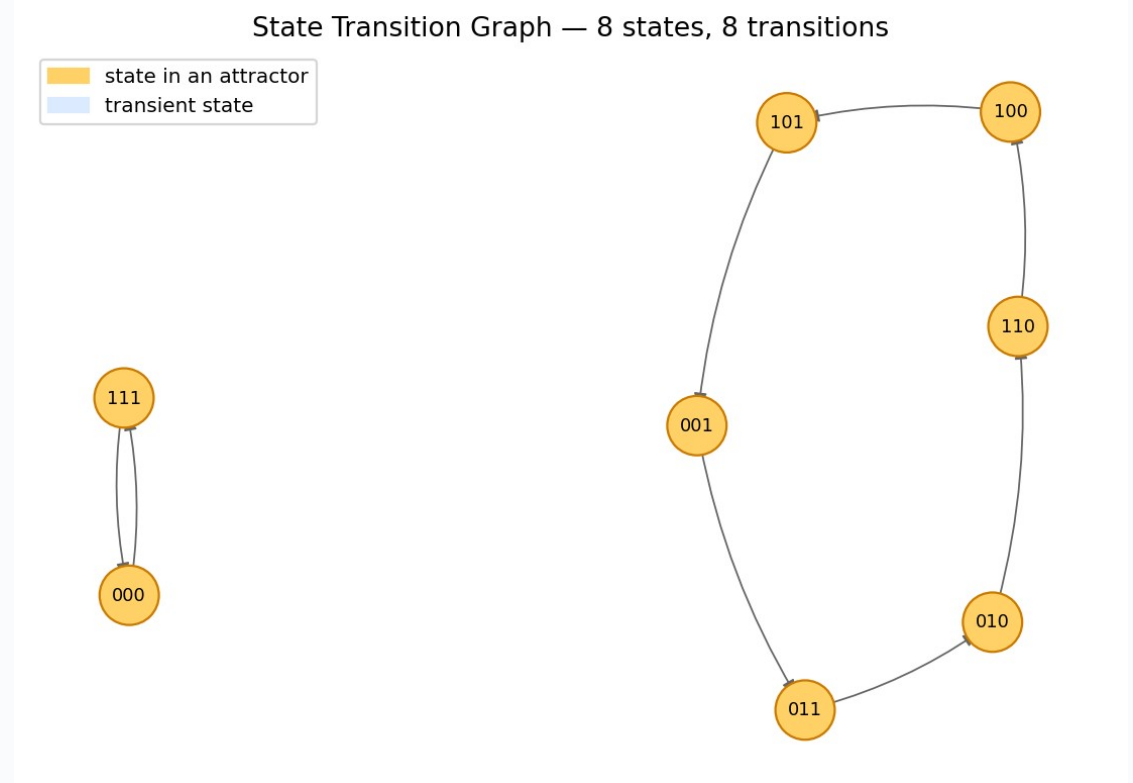
State-Transition Graph & attractors

STG

- Nodes = the 2^n possible states
- Edges = one-step transitions under the chosen update scheme

Attractor = terminal SCC

- Fixed point — singleton SCC, stable steady state
- Cyclic — SCC is a single directed cycle
- Complex — branching SCC (only under async)
- Basin of attraction = states whose trajectory ends in A



Synchronous STG of the repressilator — yellow = attractor

Presentation layer

app.py — Streamlit web UI: sidebar (load / scheme), 5 tabs (influence graph, STG, attractors, simulate, source)

Engine — bnanalyzer Python package

parser • network • stg • attractors • visualize

Storage & I/O

.bnet files (samples + user uploads) • PNG export of figures • validation through pytest

Implementation highlights

Parser & evaluator

- Tokenises both symbolic (!, &, |) and word (not, and, or) operators
- Validates: unique targets, declared inputs, syntactic correctness
- Compiles each rule with Python compile() → fast evaluation
- Sign inference: enumerate other inputs, compare output for source 0/1

```
def evaluate_all(self, state):  
    ctx = self.state_to_dict(state)  
    return tuple(  
        eval(c, {}, ctx)  
        for c in self._compiled.values())
```

STG & attractors

- build_state_transition_graph(net, scheme): full 2^n enumeration
- Sync: out-degree = 1 (deterministic)
- Async: branch on every node whose rule disagrees with current value
- Attractors = terminal SCCs in nx.condensation(g)
- Classify by structure: fixed / cyclic / complex

```
def find_attractors(g):  
    cond = nx.condensation(g)  
    return [members(c) for c in cond  
            if cond.out_degree(c) == 0]
```

Five tabs over a shared sidebar

+ available as a no-install in-browser JavaScript port ([docs/app.html](https://docs.streamlit.io/docs/app.html)) — same panels, validated to match the Python engine on every sample

1

Influence graph

Sign-coloured network drawing: + activation, - inhibition, ? non-monotonic

2

STG

Full state-transition graph; attractor states highlighted in yellow

3

Attractors

Every attractor with its kind, member states, and weak basin

4

Simulate trajectory

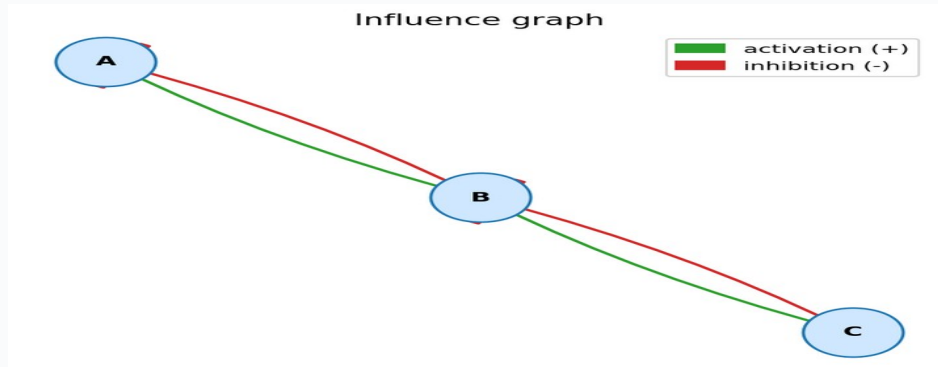
Step from any initial state; sync or async

5

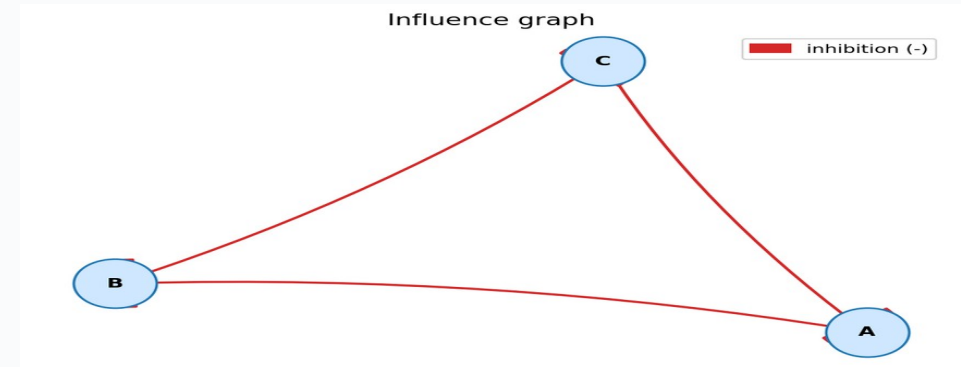
Source

Raw .bnet text with download button

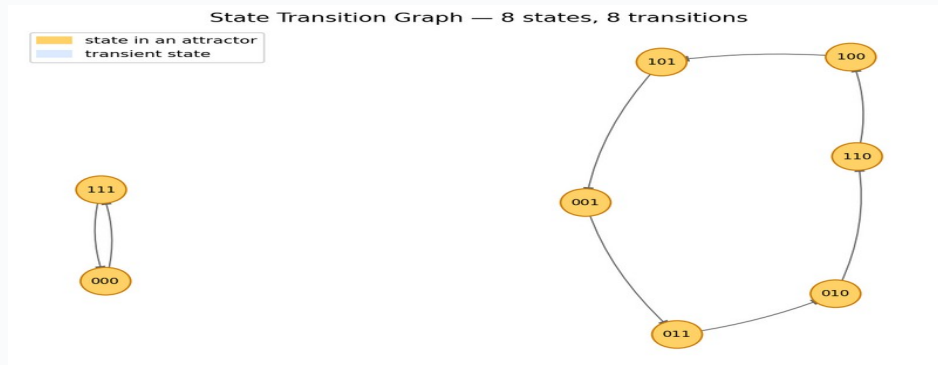
Demo & case studies



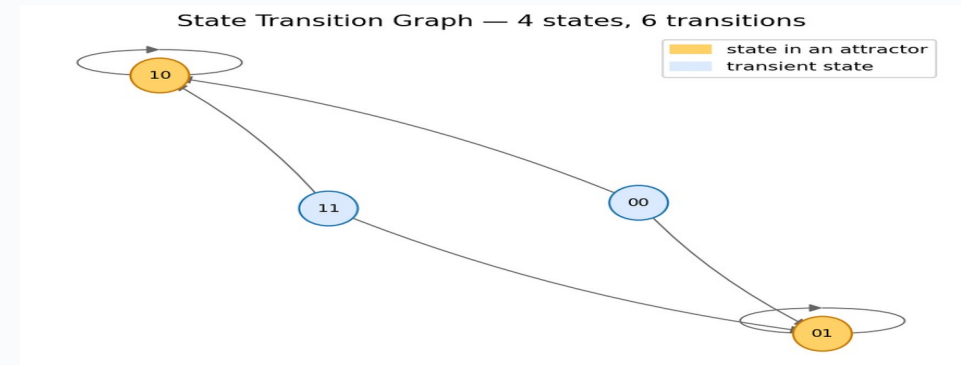
toy_switch.bnet — single length-4 cyclic attractor



repressilator.bnet — 3 inhibitions in a cycle



Sync STG — length-6 cycle + length-2 cycle



Async STG — bistable toggle, 2 fixed points

Validation against reference networks

Network	n	Sync attractors (kind / basin %)	Async attractors (kind / basin %)	Verdict
toy_switch (pedagogical)	3	1 cyclic-4 (100 %)	1 complex-8 (100 %)	OK
bistable_toggle (Gardner 2000)	2	2 fixed pts (25 % each) + 1 cyclic-2 (50 %, sync artefact)	2 fixed pts (75 % each)	MATCH bistability
repressilator (Elowitz 2000)	3	1 cyclic-6 (75 %) + 1 cyclic-2 (25 %, artefact)	1 cyclic-6 (100 %)	MATCH oscillation
fission_yeast (Davidich 2008)	9	3 cyclic-2: basins 456 / 50 / 6 (89.1 / 9.8 / 1.2 %)	1 complex-8 (100 %)	MATCH G1 dominance
mammalian_cycle (Faure 2006)	10	1 FP (G0, 50 %) + 1 cyclic-7 (50 %)	1 FP (50 %) + 1 complex-112 (50 %)	MATCH cycling region

Independently re-verified: on every (network × scheme) pair the Python reference engine and the in-browser JavaScript engine return attractors with identical kind, size, and member states.

23

pytest tests, all passing

5

reference networks bundled

<1s

runtime for the full suite

Coverage

- Parser: malformed inputs, dangling references, comments, word/symbol operators
- Engine: rule evaluation, state encoding, influence-graph signs
- STG: out-degree under sync vs async, self-loops on fixed points
- Attractors: bistable toggle, repressilator (length-6 cycle), constant rules

Limitations & future work

Current limits

- Explicit 2^n enumeration → comfortable up to ~ 16 nodes
- Static layout for the STG: hard to read past 100 states
- No probabilistic transitions
- Only synchronous and full asynchronous schemes

Roadmap

- Symbolic attractor detection (BDD / SAT) for $n \geq 30$
- Probabilistic Boolean networks
- Knock-out / over-expression perturbation analysis
- Export / import to GINsim and SBML-qual
- Interactive STG layout (D3 or pyvis)

Thank you

Questions & discussion

Tran Phan Dang Khoi • Student ID 2352626

dangkhointmk@gmail.com

GitHub repository — see README.md for the live link